交叉主管面（三面）

非技术面试准备材料 · 答题模板

1. 用一句话介绍你自己，并说说为什么适合这个岗位

我是一名做 AI 应用和 Agent 方向的工程师，比较擅长把大模型能力落到真实业务里，既做工程实现，也负责和业务方一起定义问题、验证效果。

我适合这个岗位的原因有三个。第一，我做过完整的 Agent 落地，不只是 demo，包括运行时、工具调用、动态编排、失败恢复和效果评测，知道工业场景里真正难的是什么。第二，我习惯从业务目标出发做技术方案，会先问清楚谁用、怎么衡量，而不是先选技术。第三，我有跨角色协作的经验，能把测试、业务和开发的诉求对齐，推动事情往前走。

我也很清楚自己不是单纯做算法研究的人，我的价值更多在于把技术变成业务愿意用的东西。

2. 你怎么理解这个岗位 / 这个方向的业务价值?

我的理解是，这个岗位的价值不是"做一个更聪明的模型应用"，而是用 AI 去压缩业务流程里的低效环节，并且让结果可控。

具体可以拆成三层。第一层是替代重复劳动，比如信息整理、用例生成、日志初筛，把人从机械工作里释放出来。第二层是提升判断质量，比如通过知识检索和证据链，减少遗漏和经验差异。第三层是改变工作方式，让 Agent 成为流程的一部分，而不是一个额外的工具。

但这一切有个前提：结果必须可验证、可追溯、出错可兜底。否则业务不敢真正把任务交出来，价值就停留在演示层面。所以我会把"能不能被业务持续使用"作为判断这个岗位做得好不好的核心标准，而不是看发了多少功能。

3. 讲一个你推动跨团队协作的项目，你是怎么做的?

我讲 AI4Test 项目里的一次跨角色协作。

这个项目需要测试同事提供业务规则和评审标准，也需要平台和工程侧提供环境和工具接口，但一开始大家对目标的理解并不一致：业务方希望尽快看到效果，工程侧担心不稳定，测试同事则担心增加额外工作量。

我做了三件事。第一，先把目标翻译成大家都能接受的版本：不是"用 AI 替代测试"，而是"先减少重复的用例整理工作，关键判断仍由人把关"。这个定位降低了抵触。第二，先做一个范围很小的端到端样例，让测试同事只用 review 结果，不需要额外学新工具，用实际效果建立信任。第三，把协作机制固定下来：每周一次短评审、问题清单共享、结论落到文档，明确谁负责什么、什么时候交付。

推进过程中最关键的是找到各方的共同收益，并且把对方的成本降到最低。后来这个项目从我一个人推，变成了测试同事主动提需求。

4. 当你的方案和其他团队的利益发生冲突时，你怎么处理?

我会先把"立场冲突"还原成"目标和约束的冲突"。

第一步是搞清楚对方真正在意什么。是担心稳定性、担心工作量、还是担心自己的职责被改变。很多冲突表面上是方案之争，实际是目标或考核不一致。

第二步是找共同的上位目标，比如最终都是为了让业务更快更稳地交付。在这个基础上，把方案拆成可以分别评估的部分，看能不能找到一个双方都不吃亏的交集，比如先小范围试点、分阶段上线、保留人工兜底。

第三步如果确实无法兼顾，我会把取舍和影响摆到台面上，请共同的上级或者决策方来判断，而不是在下面反复拉扯。我会提供事实和选项，不做情绪化表达。

我的原则是：可以争方案，不争面子；可以坚持，但要有依据；决定之后配合执行。

5. 如果资源被抽走、排期被压缩，你会怎么办?

我会先判断这件事是"必须保"还是"可以调"。

如果是必须保的目标，我会重新做范围拆分，保住核心链路和关键风险点，把非核心功能延后，并且明确告诉相关方延后的影响。如果目标本身也可以调整，我会把几个选项摆出来：保时间砍范围、保范围加资源、或者接受延期，让决策方选。

同时我会做风险前置。资源变少之后，最容易出问题的是那些"看起来简单但依赖多"的环节，所以我会先验证关键依赖，尽早暴露风险，而不是等到最后才发现做不完。

我不会用长期加班来掩盖资源不足，因为那会把问题变成团队的健康问题，而且不可持续。短期冲刺可以，但我会说清楚这是短期行为，并给出恢复安排。

6. 你怎么让不向你汇报的人配合你?

核心是三点：共同目标、低参与成本、可见的结果。

第一，先讲清楚这件事对对方的价值，而不是只讲我的排期。如果对方看不到收益，只靠"领导要求"推动，配合质量通常不会高。

第二，把对方需要做的事情降到最低。我会提前准备好方案、接口、数据和待确认的问题，让对方做判断而不是从零开始。能用异步文档解决的，就不开会。

第三，先做出一个小结果，让对方看到这件事真的能成。人对确定性的信任，通常来自看到过一次成功，而不是听了一次汇报。

另外我会明确责任人和时间点，重要结论留痕；遇到推不动的情况，先判断是优先级、资源还是信息问题，再决定调整方案还是升级风险。我尽量不把"推动"变成"催"，而是让对方觉得这是在共同完成一件事。

7. 信息不充分的时候，你怎么做技术决策?

我会先区分两类不确定性：一类是可以通过小成本验证的，另一类是真的一时验证不了的。

对第一类，我会设计最小验证，而不是靠讨论。比如先做一个可运行的原型、跑一组对照数据、或者小范围试点，用事实收敛分歧。

对第二类，我会按"可逆性"和"影响范围"来判断。可逆的决策快速做，先跑起来再调整；不可逆、影响面大的决策，我会多花时间对齐，保留保守方案，并明确假设和触发条件，比如"如果三个月内使用率低于某个值，就切换方案"。

我还会刻意避免过度设计。信息不足的时候，最容易犯的错误是按最复杂的未来去搭框架，结果成本很高又用不上。我的原则是先解决当前最痛的问题，把扩展点留出来，但不提前实现。

8. 你怎么看 Agent 这两年的落地风险？有哪些是被高估的?

我认为 `Agent` 的长期方向是成立的，但短期确实有一些被高估的地方。

被高估的主要是"通用自主 `Agent`"：让模型自己在开放环境里规划、调用工具、完成复杂任务，听起来很强，但真实业务里对可靠性、权限和可解释性的要求很高，一旦出错成本也高。短期内更现实的是"范围明确、结果可验证、有人兜底"的场景，比如测试辅助、信息检索、日志分析、流程里的固定环节。

主要风险有四类。第一是幻觉和不可验证，所以必须有证据链和人工确认。第二是工具副作用和权限，所以要有沙箱、审批和幂等。第三是成本和时延，所以要模型分级、缓存和预算控制。第四是评测难，很多团队只看 `demo`，没有业务标注的评测集，导致效果无法持续迭代。

所以我的判断是：机会不在"做得更像人"，而在"在明确边界内稳定地把事做完"。谁能把可靠性、评测和成本做好，谁才真正落地。

9. 如果让你从 0 搭建这个方向，前三个月你会怎么做?

我会按"选场景、跑闭环、建机制"三步走。

第一个月，先选场景和定义标准。我会和业务方一起找那种高频、规则相对明确、结果可验证的场景，明确现在的基线是什么、做好之后怎么衡量。这个阶段不急着搭大框架，重点是确认这件事值得做。

第二个月，做一个端到端的最小闭环。范围尽量窄，但链路要完整：输入、处理、工具调用、结果输出、人工兜底都要有。同时把执行轨迹和日志埋好，保证出问题能查。目标是让真实用户用起来，而不是在演示环境里好看。

第三个月，补评测和稳定性。把真实案例整理成评测集，建立回归机制；观察成功率、人工修改率、时延和成本；根据反馈决定是扩展场景还是先修稳定性。

我的原则是先窄后宽、先确定性后动态、先可验证后自动化。前三个月最重要的产出不是功能数量，而是一个能被信任的最小闭环和一套可持续迭代的机制。

10. 线上事故发生时，你怎么做对内和对外的沟通?

对内我会按"止损、同步、复盘"来处理。

第一步先止损，恢复业务优先，必要时回滚或切人工。第二步同步，把已知的影响范围、当前处置、下一次更新时间说清楚，不猜测原因，也不隐瞒。第三步复盘，等稳定后还原时间线，区分直接原因和机制原因，明确改进项、责任人和时间点。

对外沟通我会注意三点：一是及时，不要等完全查清才说，可以先给影响范围和临时方案；二是准确，不确定的部分明确说不确定，不过度承诺；三是统一口径，避免不同人对外说法不一致。

复盘我会坚持对事不对人。事故的价值在于暴露机制缺口，如果最后变成找一个人背锅，那下次大家只会隐瞒问题。真正要问的是：为什么监控没发现、为什么预案没生效、为什么同类问题还能再发生。

11. 你怎么保证方案的可扩展性和长期可维护性?

我会从三个方面控制。

第一是边界和契约。模块之间通过明确的接口交互，依赖方向是单向的，底层不反向依赖上层业务。关键数据结构有 `Schema`，配置和版本可追溯。这样后面换实现或者加能力时，不会牵一发动全身。

第二是抽象时机。我不会为了"以后可能用到"提前做很重的抽象，而是等真实场景出现第二次、第三次重复时再抽。过早抽象往往比不抽象更糟，因为它会增加理解成本，而且大概率抽错。

第三是可观测和可回滚。任何重要能力都要能被观察和定位，发布要能灰度、能回滚。长期维护最大的敌人不是代码丑，而是出了问题没人敢改。

另外我会把关键决策记录下来，说明当时为什么这么选、放弃了什么。这样后面的人不会反复推翻同样的设计，也能判断哪些前提已经变了。

12. 你怎么做知识沉淀和团队赋能?

我会把知识沉淀分成三类：能复用的、能防错的、能加速新人的。

能复用的，比如方案模板、评测集、常用工具和代码样例；能防错的，比如设计约束、`checklist`、复盘记录和常见坑；能加速新人的，比如系统全景图、上手路径和关键联系人。

形式上我不会只写大而全的文档，因为那种文档通常没人看。我更倾向于把知识嵌进流程：代码评审时检查关键约束，上线前过一遍 `checklist`，复盘后更新对应文档，新同学按任务路径学习。

分享方面，我会做短而具体的分享，比如一次只讲一个问题：为什么重试不能叠加、为什么评测集比 `demo` 重要。讲完之后希望别人能直接用在项目里，而不是听完就忘。

我认为知识沉淀的标准不是"我写了多少"，而是"别人能不能不问我，就把这件事做对"。

13. 你觉得自己目前最需要提升的是什么?

我有三个比较明确的短板。

第一是业务深度。我是技术出身，对测试和业务场景的理解更多来自项目实践，和真正的领域专家相比还有差距。我的改进方式是持续跟业务同事做具体场景的复盘，并且要求自己不仅知道怎么做，还知道为什么业务上要这么做。

第二是公开表达和影响力。我在小范围沟通没有问题，但在更大的场合做方案宣讲、推动跨团队共识时，还不够简练有力。这两年我在刻意练习，比如多做分享、提前打磨结构和例子。

第三是管理经验。我没有正式带团队，目前在项目里更多是带任务和带新人。如果岗位需要，我希望逐步补上目标管理、绩效反馈和团队建设这部分。

我不想把短板说成"我太追求完美"这种变相优点，这些确实会影响我做事的效率和边界。

14. 为什么选择我们？你打算待多久?

我选择贵司主要看三点：方向、场景和团队。

方向上是 AI 应用和 Agent，这是我已经投入并且想长期做深的方向。场景上，贵司有真实、复杂的业务，不是只做演示，这对我很有吸引力，因为 AI 的价值最终要在真实约束里体现。团队方面，我在面试里感受到大家比较务实，重视落地和工程质量，这和我的做事方式一致。

关于待多久，我是按长期来考虑的。我不希望频繁换方向，因为 Agent 这种工程能力需要在一个场景里持续积累，做一两年和做三个月完全不一样。我希望在这里至少完整经历一个从建设到稳定运行的过程，把能力做扎实。

当然前提是双方预期一致、能持续成长。如果实际工作内容和沟通时差很多，我会及时沟通，而不是拖着。

15. 基于你目前的了解，你对我们团队或业务有什么建议或疑问?

我了解得还不深，所以更多是一些想验证的疑问，不是结论。

第一，我想确认当前阶段最重要的目标是"验证价值"还是"规模化落地"。这两个阶段的打法不一样：前者应该窄场景、快验证，后者要优先补评测、稳定性和成本。

第二，我有点好奇团队目前怎么衡量 Agent 的效果。如果没有业务标注的评测集，建议尽早补，因为后面所有优化都会依赖它。

第三，我想了解业务方对这个方向的预期是什么，尤其是能接受多少人工介入。预期管理不清楚，后面很容易出现"技术做出来了但业务不敢用"的情况。

这些只是基于我有限信息的观察，实际可能有我不知道的约束，也希望能听听您的看法。

16. 你还有什么问题想问我们?

我有三个问题。

第一，从您的角度看，这个团队未来一年最需要补的能力是什么？是技术、业务理解，还是跨团队协作？

第二，这个岗位和哪些团队协作最多？目前协作中最容易卡住的地方是什么？

第三，团队怎么看待失败和试错？比如一个方向验证下来不成立，通常会怎么处理？

我也希望了解一下，您心目中这个岗位做得好的人，和做得一般的人，差别主要体现在哪里。

使用说明

- 三面更关注业务判断、跨团队影响力和长期视角，回答时尽量少讲实现细节，多讲"为什么这么做、对谁有价值、怎么衡量"。
- 第 3、4、5、9、10 题建议结合自己的真实项目准备具体例子，避免只讲原则。
- 文中【】占位符请按实际情况替换；不确定的数据不要编造，可以用范围和判断依据代替。